

Relational Databases and SQL II

Laboratórios de Sistemas e Serviços

Mário Antunes

Universidade de Aveiro

2026

SQL Database Landscape

Choosing the Right Database I

Not all RDBMS are created equal. The choice depends on the project's scale.

- **Embedded Databases (SQLite, H2):**
 - Data is stored in a local file.
 - No server setup.
 - Perfect for mobile apps, small CLI tools, and testing.
- **Server-Based Databases (MariaDB, PostgreSQL):**
 - Runs as a separate process (service).
 - Handles multiple concurrent users efficiently.
 - Industrial-grade performance and features.

Choosing the Right Database II

- **MySQL / MariaDB:**
 - The most popular for web development.
 - Fast and reliable.
- **PostgreSQL:**
 - Known for its advanced features and data integrity.
- **Oracle / SQL Server:**
 - Commercial databases for large enterprises.
 - Extremely expensive but with high-level support.

PostgreSQL: The “King” of Open Source

PostgreSQL is often the preferred choice for engineers.

- **Advanced Data Types:** JSONB (for NoSQL-like features), Arrays, Geometry.
- **Extensibility:** You can write your own functions in Python or C.
- **Standards-Compliant:** Follows the SQL standard very closely.
- **Performance:** Highly optimized for complex queries and high concurrency.

Advanced SQL: Joins and Sets

In Class 11, we saw basic Joins. Let's look at the full picture.

- **INNER JOIN:** Records with matching values in both.
- **LEFT (OUTER) JOIN:** All from left + matches from right.
- **RIGHT (OUTER) JOIN:** All from right + matches from left.
- **FULL (OUTER) JOIN:** All from both. (NULLs where no match).

Deep Dive: Joins II

- **CROSS JOIN:** Cartesian product (every row from A combined with every row from B).
- **SELF JOIN:** Joining a table to itself (e.g., employee table where manager_id references employee_id).

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;
```

Combine the results of two or more queries.

- **UNION:** Combine results and remove duplicates.
- **UNION ALL:** Combine results and keep duplicates.
- **INTERSECT:** Only rows present in both results.
- **EXCEPT (or MINUS):** Rows in the first result but not the second.

Advanced SQL: Aggregations and Window Functions

Aggregations: Summarizing Data

- **COUNT:** Number of rows.
- **SUM:** Total value.
- **AVG:** Average value.
- **MIN / MAX:** Smallest and largest.

- **GROUP BY:** Group rows that have the same values into summary rows.
- **HAVING:** Filter groups (like WHERE but for groups).

```
SELECT dept_id, COUNT(*), AVG(salary)
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 2000;
```

Common Table Expressions (CTEs)

CTEs make complex queries much more readable.

```
WITH regional_sales AS (  
    SELECT region, SUM(amount) AS total_sales  
    FROM orders  
    GROUP BY region  
)  
SELECT region, total_sales  
FROM regional_sales  
WHERE total_sales > (SELECT AVG(total_sales) FROM regional_sales);
```

- They act like “temporary views” for a single query.

Window Functions I

Window functions perform a calculation across a set of table rows that are somehow related to the current row.

- Unlike GROUP BY, they do **not** group rows into a single output row.

```
SELECT name, salary,  
       AVG(salary) OVER(PARTITION BY dept_id) as avg_dept_salary  
FROM employees;
```

- **RANK() / DENSE_RANK():** Rank rows within a partition.
- **ROW_NUMBER():** Assign a unique number to each row.

Window Functions II: Over time

“Calculate the running total of sales.”

```
SELECT date, amount,  
       SUM(amount) OVER(ORDER BY date) as running_total  
FROM sales;
```

- This is extremely powerful for analytical reports and time-series data.

Performance and Optimization

Indexes I: The Need for Speed

How does the database find one row among 10 million?

- Without an index: It scans every row (**Sequential Scan**).
- With an index: It uses a data structure (usually a B-Tree) to find it instantly.

Indexes II: Creation and Management

```
CREATE INDEX idx_student_name ON students(name);
```

- **Strategy:** Index columns used in WHERE, JOIN, and ORDER BY.
- **EXPLAIN:** Use EXPLAIN ANALYZE <query> to see how the database executes your SQL and if it's using indexes.

Scaling and Availability

The Scaling Problem

What happens when your database is too slow or too big for one server?

- **Vertical Scaling (Scale Up):** Buy a bigger server.
- **Horizontal Scaling (Scale Out):** Add more servers.

Copying data from one server (Primary) to others (Replicas).

- **Read Scaling:** Primary handles writes, Replicas handle reads.
- **Failover:** If Primary fails, a Replica becomes the new Primary.
- **Latency:** Synchronous (safe but slow) vs Asynchronous (fast but risk of loss).

Sharding: Data Partitioning

Splitting a large table into smaller pieces (shards) across different servers.

- **Strategies:**
 - **Range Sharding:** IDs 1-1000 on S1, 1001-2000 on S2.
 - **Hash Sharding:** $id \% N$ to determine server.
- **Challenge:** Joins across shards are extremely difficult.

Security and Modern Trends

NEVER concatenate user input into SQL.

BAD

```
query = "SELECT * FROM users WHERE name = '" + user_input + "';"
```

GOOD

```
cursor.execute("SELECT * FROM users WHERE name = ?", (user_input,))
```

- Use **Prepared Statements** (parameterized queries) to treat input as data, not code.

SQL vs NoSQL

Feature	SQL (Relational)	NoSQL (Non-Relational)
Schema	Rigid / Predefined	Flexible / Dynamic
Relationships	Joins (Complex)	Denormalized (Nested)
Scaling	Vertical (Mostly)	Horizontal (Easier)
Integrity	ACID compliant	BASE (Eventual Consistency)
Examples	PostgreSQL, MariaDB	MongoDB, Redis, Cassandra

Summary

Summary

- **Advanced SQL:** Master Joins, CTEs, and Window Functions.
- **Performance:** Use Indexes and EXPLAIN to optimize queries.
- **Scaling:** Use Replication for reads and Sharding for huge data.
- **Security:** Use Prepared Statements to prevent SQL Injection.
- **Choice:** Use SQL for structured data and integrity; use NoSQL for massive scale or flexible schemas.